Unit Testing Report

Project: Web Based ML Model Evaluator

Authors:

PES2UG24CS368	–	PRATHAM KUMAR	- prathamkumar041104@gmail.com
PES2UG24CS324	–	NIRUPAMA JAYARAMAN	- jnirupama06@gmail.com
PES2UG25CS824	–	SHREYAS BELLAD	- shreyasbellad2006@gmail.com
PES2UG24CS339	–	PARINITHA ACHARYA	- parinitha.acharya2006@gmail.com

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	Sheela Devi M.	sheelam@pes.edu	—

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> python -m unittest -v tests.test_unit
test_csv_blank_header_rejected (tests.test_unit.TestCsvValidation.test_csv_blank_header_rejected) ... ok
test_csv_blank_rows_ignored (tests.test_unit.TestCsvValidation.test_csv_blank_rows_ignored) ... ok
test_csv_duplicate_headers_rejected (tests.test_unit.TestCsvValidation.test_csv_duplicate_headers_rejected) ... ok
test_csv_empty_input_rejected (tests.test_unit.TestCsvValidation.test_csv_empty_input_rejected) ... ok
test_csv_extra_values_rejected (tests.test_unit.TestCsvValidation.test_csv_extra_values_rejected) ... ok
test_csv_missing_cell_is_normalized_to_empty_string (tests.test_unit.TestCsvValidation.test_csv_missing_cell_is_normalized_to_empty
_string) ... ok
test_csv_missing_header_rejected (tests.test_unit.TestCsvValidation.test_csv_missing_header_rejected) ... ok
test_csv_no_data_rows_rejected (tests.test_unit.TestCsvValidation.test_csv_no_data_rows_rejected) ... ok
test_csv_single_column_rejected (tests.test_unit.TestCsvValidation.test_csv_single_column_rejected) ... ok
test_csv_trims_cell_whitespace (tests.test_unit.TestCsvValidation.test_csv_trims_cell_whitespace) ... ok
test_csv_valid_returns_headers_and_rows (tests.test_unit.TestCsvValidation.test_csv_valid_returns_headers_and_rows) ... ok
test_auth_user_without_authorization_is_none (tests.test_unit.TestDatabaseUtilities.test_auth_user_without_authorization_is_none) .
.. ok
test_connect_enables_foreign_keys (tests.test_unit.TestDatabaseUtilities.test_connect_enables_foreign_keys) ... ok
test_init_db_creates_required_tables (tests.test_unit.TestDatabaseUtilities.test_init_db_creates_required_tables) ... ok
test_now_returns_timezone_aware_iso_timestamp (tests.test_unit.TestDatabaseUtilities.test_now_returns_timezone_aware_iso_timestamp)
 ... ok
test_owned_returns_only_matching_user_record (tests.test_unit.TestDatabaseUtilities.test_owned_returns_only_matching_user_record) .
.. ok
test_classification_accuracy_perfect (tests.test_unit.TestMetrics.test_classification_accuracy_perfect) ... ok
test_classification_accuracy_zero (tests.test_unit.TestMetrics.test_classification_accuracy_zero) ... ok
test_classification_confusion_matrix_shape (tests.test_unit.TestMetrics.test_classification_confusion_matrix_shape) ... ok
test_classification_metrics_contains_expected_keys (tests.test_unit.TestMetrics.test_classification_metrics_contains_expected_keys)
 ... ok
test_regression_error_non_negative (tests.test_unit.TestMetrics.test_regression_error_non_negative) ... ok
test_regression_metrics_contains_expected_keys (tests.test_unit.TestMetrics.test_regression_metrics_contains_expected_keys) ... ok
test_regression_perfect_error_is_zero (tests.test_unit.TestMetrics.test_regression_perfect_error_is_zero) ... ok
test_single_sample_regression_r2_is_none (tests.test_unit.TestMetrics.test_single_sample_regression_r2_is_none) ... ok
test_check_password_correct (tests.test_unit.TestPasswordUtilities.test_check_password_correct) ... ok
test_check_password_incorrect (tests.test_unit.TestPasswordUtilities.test_check_password_incorrect) ... ok
test_check_password_malformed_empty (tests.test_unit.TestPasswordUtilities.test_check_password_malformed_empty) ... ok
test_check_password_malformed_extra_separator (tests.test_unit.TestPasswordUtilities.test_check_password_malformed_extra_separator)
 ... ok
test_check_password_malformed_no_separator (tests.test_unit.TestPasswordUtilities.test_check_password_malformed_no_separator) ... o
k
test_hash_accepts_supplied_salt (tests.test_unit.TestPasswordUtilities.test_hash_accepts_supplied_salt) ... ok
test_hash_returns_salt_separator_digest (tests.test_unit.TestPasswordUtilities.test_hash_returns_salt_separator_digest) ... ok
test_hash_uses_unique_random_salts (tests.test_unit.TestPasswordUtilities.test_hash_uses_unique_random_salts) ... ok

----------------------------------------------------------------------
Ran 32 tests in 1.661s

OK
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final>
```

1. INTRODUCTION

1.1 Project Overview

The Web-Based ML Model Evaluator (ModelLab) is a web-based application designed to simplify and streamline the process of developing, training, and evaluating machine learning models. The platform provides an integrated environment where users can upload datasets, explore data, perform preprocessing operations, configure machine learning experiments, train models, and evaluate their performance through appropriate evaluation metrics.

The primary objective of ModelLab is to provide users with a centralized platform for experimenting with different machine learning algorithms without requiring them to independently implement the entire model development and evaluation workflow. It supports both classification and regression tasks, allowing users to select suitable algorithms based on the nature of their datasets and prediction objectives.

The application integrates frontend and backend components to provide a structured workflow for machine learning experimentation. The frontend facilitates user interaction, while the backend handles data processing, model execution, and evaluation operations. Through this integration, ModelLab aims to make machine learning experimentation more organized, accessible, and efficient.

Since the application consists of multiple functionalities and interconnected components, it is important to verify that its individual functions operate correctly. Unit testing is therefore performed to identify errors at the individual component level before relying on the complete application workflow.

1.2 Introduction to Unit Testing

Unit testing is a software testing technique in which individual units or components of an application are tested independently to verify that they perform their intended operations correctly. A unit generally refers to the smallest independently testable part of a software system, such as a function, method, or individual module. The primary purpose of unit testing is to detect logical errors and unexpected behaviour at an early stage of software development. Instead of testing the entire application at once, each individual function is supplied with specific inputs, and its actual output is compared against the expected output.

Unit testing helps ensure that individual components behave correctly under different input conditions, including valid inputs, invalid inputs, boundary conditions, and exceptional cases. It also makes debugging easier because errors can be isolated to the specific function or component being tested.

In the ModelLab application, unit testing is used to verify the correctness of individual backend functions and application-level operations involved in the machine learning workflow. This includes checking data-related operations, preprocessing functionality, model-related operations, and other independently testable functionalities implemented within the application.

1.3 Unit Testing Strategy and Implementation

For implementing unit testing in ModelLab, we adopted a structured, function-oriented testing strategy using the Pytest framework in Python. The objective was to validate individual application functions independently by providing controlled inputs and verifying their corresponding outputs.

The unit testing implementation was organized within a dedicated tests directory. The test cases were defined in the test_unit.py file, with supporting utilities maintained separately wherever required. This modular organization makes the testing implementation easier to understand, execute, maintain, and extend.

The following strategy was adopted for implementing unit testing:
1. Identification of Testable Units: Individual functions and operations within the application were identified based on their responsibilities and expected behaviour.

2. Test Case Design: Separate test cases were designed for the identified functions to verify their expected functionality under different input conditions.

3. Input Preparation: Appropriate test inputs were supplied to each function, including valid inputs and relevant edge cases wherever applicable.

4. Expected Output Verification: The actual output generated by each function was compared with the expected result using assertions provided by Pytest.

5. Exception and Error Handling: Relevant test cases were designed to verify whether functions handle invalid inputs and unexpected conditions appropriately.

6. Automated Test Execution: All unit test cases were executed using the Pytest framework, allowing the results of individual tests to be recorded automatically.

7. Result Analysis: The execution results were examined to identify successful test cases, failures, and potential issues requiring correction.

The complete unit testing suite was implemented using the Python testing framework and integrated into the consolidated testing execution script, run_all_tests.py. This allows unit testing to be executed alongside the other testing categories within the project.

Tools and technologies used:

Tool / Technology	Purpose
Python	Programming language used for implementing test cases
Pytest	Framework used to define and execute unit tests
Assertions	Verify actual outputs against expected outputs
PowerShell	Execute the testing suite in the local environment
HTML Reports	Present generated testing results in a readable format

The unit testing implementation consists of 32 test cases, designed to validate individual functionalities of the ModelLab application. These test cases are executed independently as part of the unit testing stage, and their results are recorded for further analysis.

2. UNIT TESTING TEST CASES

2.1 Overview of Unit Test Cases

A total of 32 unit test cases have been implemented in ModelLab to verify the correctness of individual backend functions and database utilities.

The test cases are implemented in Python using the unittest framework and are organized into four test classes within tests/test_unit.py.

The test cases cover the following functional areas:

Test class	Test cases	Purpose
TestPasswordUtilities	T1–T8	Password hashing and verification
TestCsvValidation	T9–T19	CSV input validation and processing
TestMetrics	T20–T27	Classification and regression metrics
TestDatabaseUtilities	T28–T32	Database operations and authentication
Total	32	

The implementation uses assertions to compare actual function behaviour with expected results. It also uses mocking and an isolated database helper to test selected operations without depending on the normal application database.

2.2 Detailed Unit Test Cases

2.2.1 Password Utilities Testing

Objective: To verify the correctness and reliability of password hashing and password verification functions implemented in the backend.
These test cases examine password hash generation, salt handling, successful password verification, incorrect passwords and malformed password hashes.

Test Case T1 – Hash Returns Salt Separator and Digest

- Function tested: server.hash_pw()
- Input: "StrongPass123"
- Testing procedure: Generate a password hash and split the returned value using the $ separator.
- Expected output: The generated value must contain exactly two parts: salt and digest.
- Validation: self.assertEqual(len(value.split("$")), 2)

Test Case T2 – Hash Uses Unique Random Salts

- Function tested: server.hash_pw()
- Input: The same password, "same", supplied twice.
- Testing procedure: Generate two password hashes using the same password and compare them.
- Expected output: Both generated hashes must be different because independent random salts are used.
- Validation: self.assertNotEqual(server.hash_pw("same"), server.hash_pw("same"))

Test Case T3 – Hash Accepts Supplied Salt

- Function tested: server.hash_pw()
- Input: Password "pw" and fixed salt "fixed".
- Testing procedure: Generate the hash twice using the same password and explicitly supplied salt.
- Expected output: Both generated hashes must be identical.
- Validation: self.assertEqual(server.hash_pw("pw", "fixed"), server.hash_pw("pw", "fixed"))

Test Case T4 – Correct Password Verification

- Function tested: server.check_pw()
- Input: Password "abc12345" and its generated hash.
- Testing procedure: Verify the original password against its generated hash.
- Expected output: The password verification function must return True.
- Validation: self.assertTrue(...)

Test Case T5 – Incorrect Password Verification

- Function tested: server.check_pw()

- Input: Original password "abc12345" and incorrect password "wrong123".
- Testing procedure: Attempt to verify the incorrect password against the original password hash.
- Expected output: The function must return False.
- Validation: self.assertFalse(...)

Test Case T6 – Malformed Empty Password Hash

- Function tested: server.check_pw()
- Input: Password "pw" and an empty hash string.
- Testing procedure: Pass an empty stored hash to the verification function.
- Expected output: The function must safely return False.
- Validation: self.assertFalse(server.check_pw("pw", ""))

Test Case T7 – Malformed Hash Without Separator

- Function tested: server.check_pw()
- Input: Password "pw" and malformed hash "malformed".
- Testing procedure: Verify a password against a hash that does not contain the expected separator.
- Expected output: The function must return False.
- Validation: self.assertFalse(server.check_pw("pw", "malformed"))

Test Case T8 – Malformed Hash With Extra Separator

- Function tested: server.check_pw()
- Input: Password "pw" and hash "salt$digest$extra".
- Testing procedure: Pass a hash containing more separators than the expected format.
- Expected output: The function must return False.
- Validation: self.assertFalse(server.check_pw("pw", "salt$digest$extra"))

Code screenshot

```
PS C:\Users\HP> cd "D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final"
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> Get-Content ".\tests\test_unit.py" | Select-Object -First 65
"""Unit tests: isolated functions, validation rules, metrics and persistence helpers."""
import unittest
from unittest.mock import Mock
from tests.helpers import isolated_database, CLASSIFICATION_CSV
import server


class TestPasswordUtilities(unittest.TestCase):
    def test_hash_returns_salt_separator_digest(self):
        value = server.hash_pw("StrongPass123")
        self.assertEqual(len(value.split("$")), 2)

    def test_hash_uses_unique_random_salts(self):
        self.assertNotEqual(server.hash_pw("same"), server.hash_pw("same"))

    def test_hash_accepts_supplied_salt(self):
        self.assertEqual(server.hash_pw("pw", "fixed"), server.hash_pw("pw", "fixed"))

    def test_check_password_correct(self):
        self.assertTrue(server.check_pw("abc12345", server.hash_pw("abc12345")))

    def test_check_password_incorrect(self):
        self.assertFalse(server.check_pw("wrong123", server.hash_pw("abc12345")))

    def test_check_password_malformed_empty(self):
        self.assertFalse(server.check_pw("pw", ""))

    def test_check_password_malformed_no_separator(self):
        self.assertFalse(server.check_pw("pw", "malformed"))

    def test_check_password_malformed_extra_separator(self):
        self.assertFalse(server.check_pw("pw", "salt$digest$extra"))


class TestCsvValidation(unittest.TestCase):
    def test_csv_valid_returns_headers_and_rows(self):
        headers, rows = server.csv_data("name,age\nA,1\nB,2\n")
        self.assertEqual(headers, ["name", "age"])
        self.assertEqual(len(rows), 2)

    def test_csv_missing_cell_is_normalized_to_empty_string(self):
        _, rows = server.csv_data("name,age\nAlice,\n")
        self.assertEqual(rows[0], {"name": "Alice", "age": ""})

    def test_csv_trims_cell_whitespace(self):
        _, rows = server.csv_data("name,age\n A , 20 \n")
        self.assertEqual(rows[0], {"name": "A", "age": "20"})

    def test_csv_empty_input_rejected(self):
        with self.assertRaisesRegex(ValueError, "empty"):
            server.csv_data("  \n")

    def test_csv_missing_header_rejected(self):
        with self.assertRaises(ValueError):
            server.csv_data("")

    def test_csv_single_column_rejected(self):
        with self.assertRaisesRegex(ValueError, "at least two"):
            server.csv_data("name\nAlice\n")

    def test_csv_duplicate_headers_rejected(self):
        with self.assertRaisesRegex(ValueError, "unique"):
            server.csv_data("name,name\nA,B\n")

    def test_csv_blank_header_rejected(self):
```

Figure 2.1 – Password utility unit test implementations (T1–T8).

Explanation: The code verifies password hash generation, random salt uniqueness, fixed salt consistency, correct and incorrect password verification, and handling of malformed stored hashes.

2.2.2 CSV Validation Testing

Objective: To verify that the CSV processing function correctly reads valid CSV input, extracts headers and rows, normalizes data, and rejects invalid CSV structures.

These test cases validate the behaviour of server.csv_data() under valid and invalid input conditions.

Test Case T9 – Valid CSV Returns Headers and Rows

- Function tested: server.csv_data()
- Input: name,age with two data rows: A,1 and B,2.
- Testing procedure: Supply a valid CSV string and extract its headers and rows.
- Expected output: Headers must be ["name", "age"], and two data rows must be returned.
- Validation: Header equality and row count of 2.

Test Case T10 – Missing Cell Is Normalized to Empty String

- Function tested: server.csv_data()
- Input: name,age with the row Alice,.
- Testing procedure: Process a CSV row containing a missing cell.
- Expected output: The missing age value must be represented as an empty string.
- Validation: {"name": "Alice", "age": ""}

Test Case T11 – CSV Trims Cell Whitespace

- Function tested: server.csv_data()
- Input: A and 20 as CSV cell values.
- Testing procedure: Process CSV data containing leading and trailing spaces.
- Expected output: Values must be normalized to "A" and "20".
- Validation: Equality of the processed row with the expected dictionary.

Test Case T12 – Empty CSV Input Rejected

- Function tested: server.csv_data()
- Input: A whitespace-only CSV string.
- Testing procedure: Pass empty or whitespace-only input to the CSV processing function.
- Expected output: A ValueError containing the message "empty" must be raised.
- Validation: self.assertRaisesRegex(ValueError, "empty")

Test Case T13 – Missing Header Rejected

- Function tested: server.csv_data()
- Input: An empty CSV string.
- Testing procedure: Attempt to process CSV data without a header.
- Expected output: The function must raise a ValueError.
- Validation: self.assertRaises(ValueError)

Test Case T14 – Single Column CSV Rejected

- Function tested: server.csv_data()
- Input: A CSV containing only one column, name, with the value Alice.
- Testing procedure: Supply a CSV that does not satisfy the minimum column requirement.
- Expected output: A ValueError containing "at least two" must be raised.
- Validation: self.assertRaisesRegex(ValueError, "at least two")

Test Case T15 – Duplicate Headers Rejected

- Function tested: server.csv_data()
- Input: CSV header name,name.
- Testing procedure: Process a CSV containing duplicate column names.
- Expected output: A ValueError containing "unique" must be raised.
- Validation: self.assertRaisesRegex(ValueError, "unique")

Test Case T16 – Blank Header Rejected

- Function tested: server.csv_data()
- Input: CSV header name, containing a blank second column name.
- Testing procedure: Process a CSV containing an empty or whitespace-only header.
- Expected output: A ValueError containing "unique" must be raised.
- Validation: self.assertRaisesRegex(ValueError, "unique")

Test Case T17 – CSV Without Data Rows Rejected

- Function tested: server.csv_data()
- Input: CSV containing only the header name,age.
- Testing procedure: Attempt to process a CSV with valid headers but no data records.
- Expected output: A ValueError containing "no data" must be raised.
- Validation: self.assertRaisesRegex(ValueError, "no data")

Test Case T18 – Blank CSV Rows Ignored

- Function tested: server.csv_data()

- Input: Two valid data rows separated by a blank row.
- Testing procedure: Process CSV input containing an empty row between valid records.
- Expected output: The blank row must be ignored, and exactly two valid rows must be returned.
- Validation: self.assertEqual(len(rows), 2)

Test Case T19 – Extra CSV Values Rejected

- Function tested: server.csv_data()
- Input: Two headers (a,b) but a data row containing three values (1,2,3).
- Testing procedure: Process a CSV record containing more values than the defined headers.
- Expected output: A ValueError containing "extra values" must be raised.
- Validation: self.assertRaisesRegex(ValueError, "extra values")

Code screenshot

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> Get-Content ".\tests\test_unit.py" | Select-Object -Skip 64 -First 110
    def test_csv_blank_header_rejected(self):
        with self.assertRaisesRegex(ValueError, "unique"):
            server.csv_data("name, \nA,B\n")

    def test_csv_no_data_rows_rejected(self):
        with self.assertRaisesRegex(ValueError, "no data"):
            server.csv_data("name,age\n")

    def test_csv_blank_rows_ignored(self):
        _, rows = server.csv_data("name,age\nA,1\n,\nB,2\n")
        self.assertEqual(len(rows), 2)

    def test_csv_extra_values_rejected(self):
        with self.assertRaisesRegex(ValueError, "extra values"):
            server.csv_data("a,b\n1,2,3\n")


class TestMetrics(unittest.TestCase):
    def test_classification_metrics_contains_expected_keys(self):
        result = server.metrics_for(["a", "b"], ["a", "b"], "classification")
        self.assertTrue({"accuracy", "precision_weighted", "recall_weighted",
                         "f1_weighted", "confusion_matrix"} <= result.keys())

    def test_classification_accuracy_perfect(self):
        result = server.metrics_for([0, 1, 1], [0, 1, 1], "classification")
        self.assertEqual(result["accuracy"], 1.0)

    def test_classification_accuracy_zero(self):
        result = server.metrics_for([0, 0], [1, 1], "classification")
        self.assertEqual(result["accuracy"], 0.0)

    def test_classification_confusion_matrix_shape(self):
        result = server.metrics_for(["a", "b", "a"], ["a", "b", "b"], "classification")
        self.assertEqual(len(result["confusion_matrix"]), 2)

    def test_regression_metrics_contains_expected_keys(self):
        result = server.metrics_for([1, 2, 3], [1, 2, 3], "regression")
        self.assertEqual(set(result), {"mae", "rmse", "r2"})

    def test_regression_perfect_error_is_zero(self):
        result = server.metrics_for([1, 2, 3], [1, 2, 3], "regression")
        self.assertEqual(result["mae"], 0.0)
        self.assertEqual(result["rmse"], 0.0)
        self.assertEqual(result["r2"], 1.0)

    def test_regression_error_non_negative(self):
        result = server.metrics_for([1, 2, 3], [2, 2, 4], "regression")
        self.assertGreaterEqual(result["mae"], 0)
        self.assertGreaterEqual(result["rmse"], 0)

    def test_single_sample_regression_r2_is_none(self):
        result = server.metrics_for([1], [2], "regression")
        self.assertIsNone(result["r2"])


class TestDatabaseUtilities(unittest.TestCase):
    def test_now_returns_timezone_aware_iso_timestamp(self):
        from datetime import datetime
        value = datetime.fromisoformat(server.now())
        self.assertIsNotNone(value.tzinfo)
```

```
    def test_init_db_creates_required_tables(self):
        with isolated_database():
            with server.connect() as conn:
                names = {r["name"] for r in conn.execute(
                    "SELECT name FROM sqlite_master WHERE type='table'")}
            self.assertTrue({"users", "sessions", "projects", "datasets", "experiments"} <= names)

    def test_connect_enables_foreign_keys(self):
        with isolated_database():
            with server.connect() as conn:
                self.assertEqual(conn.execute("PRAGMA foreign_keys").fetchone()[0], 1)

    def test_owned_returns_only_matching_user_record(self):
        with isolated_database():
            with server.connect() as conn:
                cur = conn.execute("INSERT INTO users(name,email,pw_hash,created) VALUES(?,?,?,?)",
                                   ("A", "a@test.local", "x", server.now()))
                uid = cur.lastrowid
                cur = conn.execute("INSERT INTO projects(user_id,name,description,created) VALUES(?,?,?,?)",
                                   (uid, "P", "", server.now()))
                pid = cur.lastrowid
                self.assertIsNotNone(server.owned(conn, "projects", pid, uid))
                self.assertIsNone(server.owned(conn, "projects", pid, uid + 1))

    def test_auth_user_without_authorization_is_none(self):
        handler = Mock()
        handler.headers = {"Authorization": ""}
        self.assertIsNone(server.auth_user(handler))
```

Figure 2.2 – CSV validation unit test implementations (T9–T19).

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content = Get-Content ".\tests\test_unit.py"
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $start = ($content | Select-String '^class TestCsvValidation').L
ineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $end = ($content | Select-String '^class TestMetrics').LineNumbe
r
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content[($start - 1)..($end - 2)]
class TestCsvValidation(unittest.TestCase):
    def test_csv_valid_returns_headers_and_rows(self):
        headers, rows = server.csv_data("name,age\nA,1\nB,2\n")
        self.assertEqual(headers, ["name", "age"])
        self.assertEqual(len(rows), 2)

    def test_csv_missing_cell_is_normalized_to_empty_string(self):
        _, rows = server.csv_data("name,age\nAlice,\n")
        self.assertEqual(rows[0], {"name": "Alice", "age": ""})

    def test_csv_trims_cell_whitespace(self):
        _, rows = server.csv_data("name,age\n A , 20 \n")
        self.assertEqual(rows[0], {"name": "A", "age": "20"})

    def test_csv_empty_input_rejected(self):
        with self.assertRaisesRegex(ValueError, "empty"):
            server.csv_data("  \n")

    def test_csv_missing_header_rejected(self):
        with self.assertRaises(ValueError):
            server.csv_data("")

    def test_csv_single_column_rejected(self):
        with self.assertRaisesRegex(ValueError, "at least two"):
            server.csv_data("name\nAlice\n")

    def test_csv_duplicate_headers_rejected(self):
        with self.assertRaisesRegex(ValueError, "unique"):
            server.csv_data("name,name\nA,B\n")

    def test_csv_blank_header_rejected(self):
        with self.assertRaisesRegex(ValueError, "unique"):
            server.csv_data("name, \nA,B\n")

    def test_csv_no_data_rows_rejected(self):
        with self.assertRaisesRegex(ValueError, "no data"):
            server.csv_data("name,age\n")

    def test_csv_blank_rows_ignored(self):
        _, rows = server.csv_data("name,age\nA,1\n,\nB,2\n")
        self.assertEqual(len(rows), 2)

    def test_csv_extra_values_rejected(self):
        with self.assertRaisesRegex(ValueError, "extra values"):
            server.csv_data("a,b\n1,2,3\n")


PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final>
```

Explanation: The code validates CSV reading and processing, including header extraction, missing values, whitespace normalization, duplicate headers, empty datasets, blank rows and extra values. It ensures that invalid CSV structures are rejected appropriately.

2.2.3 Classification and Regression Metrics Testing

Objective: To verify that the backend correctly calculates and returns evaluation metrics for classification and regression machine learning tasks.

These test cases examine metric availability, accuracy calculation, confusion matrix structure, regression error calculations and special cases involving limited input samples.

Test Case T20 – Classification Metrics Contain Expected Keys

- Function tested: server.metrics_for()
- Input: Actual labels ["a", "b"], predicted labels ["a", "b"], task type "classification".
- Testing procedure: Calculate classification metrics and inspect the returned dictionary.
- Expected output: The result must contain accuracy, weighted precision, weighted recall, weighted F1-score and confusion matrix.
- Validation: A subset check against the expected metric keys.

Test Case T21 – Classification Accuracy With Perfect Predictions

- Function tested: server.metrics_for()
- Input: Actual labels [0, 1, 1] and predicted labels [0, 1, 1].
- Testing procedure: Calculate accuracy when all predictions match the actual labels.
- Expected output: Accuracy must be 1.0.
- Validation: self.assertEqual(result["accuracy"], 1.0)

Test Case T22 – Classification Accuracy With Zero Correct Predictions

- Function tested: server.metrics_for()
- Input: Actual labels [0, 0] and predicted labels [1, 1].
- Testing procedure: Calculate accuracy when every prediction is incorrect.
- Expected output: Accuracy must be 0.0.
- Validation: self.assertEqual(result["accuracy"], 0.0)

Test Case T23 – Classification Confusion Matrix Shape

- Function tested: server.metrics_for()
- Input: Actual labels ["a", "b", "a"] and predicted labels ["a", "b", "b"].

- Testing procedure: Generate the classification confusion matrix and examine its dimensions.
- Expected output: The confusion matrix must contain two rows, corresponding to the two classes.
- Validation: self.assertEqual(len(result["confusion_matrix"]), 2)

Test Case T24 – Regression Metrics Contain Expected Keys

- Function tested: server.metrics_for()
- Input: Actual values [1, 2, 3], predicted values [1, 2, 3], task type "regression".
- Testing procedure: Calculate regression evaluation metrics.
- Expected output: The result must contain exactly mae, rmse and r2.
- Validation: self.assertEqual(set(result), {"mae", "rmse", "r2"})

Test Case T25 – Perfect Regression Predictions Produce Zero Error

- Function tested: server.metrics_for()
- Input: Actual values [1, 2, 3] and predicted values [1, 2, 3].
- Testing procedure: Calculate regression metrics when actual and predicted values are identical.
- Expected output: MAE and RMSE must be 0.0, while R^2 must be 1.0.
- Validation: Equality checks for all three metrics.

Test Case T26 – Regression Errors Are Non-Negative

- Function tested: server.metrics_for()
- Input: Actual values [1, 2, 3] and predicted values [2, 2, 4].
- Testing procedure: Calculate regression errors for predictions that differ from actual values.
- Expected output: Both MAE and RMSE must be greater than or equal to zero.
- Validation: self.assertGreaterEqual() for both metrics.

Test Case T27 – Single-Sample Regression R^2 Is None

- Function tested: server.metrics_for()
- Input: Actual value [1] and predicted value [2].
- Testing procedure: Calculate regression metrics using only one sample.
- Expected output: R^2 must be None, as the implementation handles the single-sample case this way.
- Validation: self.assertIsNone(result["r2"])

Code screenshot

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content = Get-Content ".\tests\test_unit.py"
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $start = ($content | Select-String '^class TestMetrics').LineNum
ber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $end = ($content | Select-String '^class TestDatabaseUtilities')
.LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content[($start - 1)..($end - 2)]
class TestMetrics(unittest.TestCase):
    def test_classification_metrics_contains_expected_keys(self):
        result = server.metrics_for(["a", "b"], ["a", "b"], "classification")
        self.assertTrue({"accuracy", "precision_weighted", "recall_weighted",
                         "f1_weighted", "confusion_matrix"} <= result.keys())

    def test_classification_accuracy_perfect(self):
        result = server.metrics_for([0, 1, 1], [0, 1, 1], "classification")
        self.assertEqual(result["accuracy"], 1.0)

    def test_classification_accuracy_zero(self):
        result = server.metrics_for([0, 0], [1, 1], "classification")
        self.assertEqual(result["accuracy"], 0.0)

    def test_classification_confusion_matrix_shape(self):
        result = server.metrics_for(["a", "b", "a"], ["a", "b", "b"], "classification")
        self.assertEqual(len(result["confusion_matrix"]), 2)

    def test_regression_metrics_contains_expected_keys(self):
        result = server.metrics_for([1, 2, 3], [1, 2, 3], "regression")
        self.assertEqual(set(result), {"mae", "rmse", "r2"})

    def test_regression_perfect_error_is_zero(self):
        result = server.metrics_for([1, 2, 3], [1, 2, 3], "regression")
        self.assertEqual(result["mae"], 0.0)
        self.assertEqual(result["rmse"], 0.0)
        self.assertEqual(result["r2"], 1.0)

    def test_regression_error_non_negative(self):
        result = server.metrics_for([1, 2, 3], [2, 2, 4], "regression")
        self.assertGreaterEqual(result["mae"], 0)
        self.assertGreaterEqual(result["rmse"], 0)

    def test_single_sample_regression_r2_is_none(self):
        result = server.metrics_for([1], [2], "regression")
        self.assertIsNone(result["r2"])


PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final>
```

Figure 2.3 – Classification and regression metrics unit test implementations (T20–T27).

Explanation: The code verifies the availability and correctness of classification and regression metrics, including accuracy, confusion matrix dimensions, MAE, RMSE and R^2. It also checks perfect predictions, incorrect predictions and the single-sample regression case.

2.2.4 Database Utilities Testing

Objective: To verify the correctness of database initialization, timestamp generation, database connection configuration, record ownership validation and authentication handling.

These test cases use an isolated database environment for selected database operations, ensuring that the tests do not interfere with the application's normal database.

Test Case T28 – Time Function Returns Timezone-Aware ISO Timestamp

- Function tested: server.now()
- Input: No explicit input; the function generates the current timestamp.
- Testing procedure: Call the function and parse the returned timestamp using datetime.fromisoformat().
- Expected output: The returned timestamp must contain timezone information.
- Validation: self.assertIsNotNone(value.tzinfo)

Test Case T29 – Database Initialization Creates Required Tables

- Function tested: server.init_db()
- Input: An isolated database environment.
- Testing procedure: Initialize the database, retrieve the table names from SQLite's sqlite_master, and compare them with the required table names.
- Expected output: The database must contain the following tables:
  - users
  - sessions
  - projects
  - datasets
  - experiments
- Validation: The required table names must be present in the retrieved table collection.

Test Case T30 – Database Connection Enables Foreign Keys

- Function tested: server.connect()
- Input: An isolated database connection.
- Testing procedure: Establish a database connection and execute PRAGMA foreign_keys to inspect the foreign-key configuration.
- Expected output: Foreign-key enforcement must be enabled, returning 1.
- Validation: self.assertEqual(..., 1)

Test Case T31 – Ownership Validation Returns Only Matching User Record

- Function tested: server.owned()
- Input: A project record associated with a specific user ID.
- Testing procedure: Insert a user and an associated project into the isolated database. Then check ownership using the correct user ID and an incorrect user ID.
- Expected output:
  - The correct owner must receive the matching project record.
  - A different user must not receive the project record.

- Validation: self.assertIsNotNone() for the matching owner and self.assertIsNone() for the incorrect owner.

Test Case T32 – Authentication Without Authorization Returns None

- Function tested: server.auth_user()
- Input: A mocked request handler with an empty Authorization header.
- Testing procedure: Pass the mocked handler to the authentication function without providing authorization credentials.
- Expected output: The authentication function must return None.
- Validation: self.assertIsNone(server.auth_user(handler))

Code screenshot

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content = Get-Content ".\tests\test_unit.py"
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $start = ($content | Select-String '^class TestDatabaseUtilities
').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content[($start - 1)..($content.Length - 1)]
class TestDatabaseUtilities(unittest.TestCase):
    def test_now_returns_timezone_aware_iso_timestamp(self):
        from datetime import datetime
        value = datetime.fromisoformat(server.now())
        self.assertIsNotNone(value.tzinfo)

    def test_init_db_creates_required_tables(self):
        with isolated_database():
            with server.connect() as conn:
                names = {r["name"] for r in conn.execute(
                    "SELECT name FROM sqlite_master WHERE type='table'")}
            self.assertTrue({"users", "sessions", "projects", "datasets", "experiments"} <= names)

    def test_connect_enables_foreign_keys(self):
        with isolated_database():
            with server.connect() as conn:
                self.assertEqual(conn.execute("PRAGMA foreign_keys").fetchone()[0], 1)

    def test_owned_returns_only_matching_user_record(self):
        with isolated_database():
            with server.connect() as conn:
                cur = conn.execute("INSERT INTO users(name,email,pw_hash,created) VALUES(?,?,?,?)",
                                   ("A", "a@test.local", "x", server.now()))
                uid = cur.lastrowid
                cur = conn.execute("INSERT INTO projects(user_id,name,description,created) VALUES(?,?,?,?)",
                                   (uid, "P", "", server.now()))
                pid = cur.lastrowid
                self.assertIsNotNone(server.owned(conn, "projects", pid, uid))
                self.assertIsNone(server.owned(conn, "projects", pid, uid + 1))

    def test_auth_user_without_authorization_is_none(self):
        handler = Mock()
        handler.headers = {"Authorization": ""}
        self.assertIsNone(server.auth_user(handler))


if __name__ == "__main__":
    unittest.main(verbosity=2)
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> -|
```

Figure 2.4 – Database utilities unit test implementations (T28–T32).

Explanation: The code verifies timestamp generation, database initialization, foreign-key configuration, record ownership validation and handling of requests without authorization credentials.

2.3 Summary of Unit Test Case Implementation

The 32 unit test cases cover four major functional areas of the ModelLab backend. Each test case validates a specific operation using controlled inputs and Python assertions.

Test case range	Functional area	Number of tests
T1–T8	Password utilities	8
T9–T19	CSV validation	11
T20–T27	Classification and regression metrics	8
T28–T32	Database utilities	5
Total		32

The implementation also uses:

- unittest.TestCase for organizing individual test cases.
- unittest.mock.Mock for simulating an HTTP request handler.
- isolated_database for testing database operations independently.
- Python assertions for validating expected behaviour.

Conclusion of Section 2

The unit testing implementation provides individual validation of important ModelLab backend operations, covering password utilities, CSV processing, machine learning evaluation metrics, database operations and authentication handling. These test cases help identify functional errors at the individual unit level before evaluating the interaction of components within the complete application.

3. Unit Testing Results and Outputs

3.1 Unit Testing Execution

The unit testing suite was executed using the run_all_tests.py script from the ModelLab project directory. The execution covered all 32 unit test cases implemented in the project.

The test cases were executed across four categories: Password Utilities, CSV Validation, Classification and Regression Metrics, and Database Utilities.

```
PS C:\Users\HP> cd "D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final"
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> python run_all_tests.py
================================================================================
MODELLAB - COMPLETE SOFTWARE TESTING
================================================================================
Project directory: D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final
Started: 2026-11-11T12:00:35+05:30

==================== Unit testing ====================
test_csv_blank_header_rejected (tests.test_unit.TestCsvValidation.test_csv_blank_header_rejected) ... ok
test_csv_blank_rows_ignored (tests.test_unit.TestCsvValidation.test_csv_blank_rows_ignored) ... ok
test_csv_duplicate_headers_rejected (tests.test_unit.TestCsvValidation.test_csv_duplicate_headers_rejected) ... ok
test_csv_empty_input_rejected (tests.test_unit.TestCsvValidation.test_csv_empty_input_rejected) ... ok
test_csv_extra_values_rejected (tests.test_unit.TestCsvValidation.test_csv_extra_values_rejected) ... ok
test_csv_missing_cell_is_normalized_to_empty_string (tests.test_unit.TestCsvValidation.test_csv_missing_cell_is_normalized_to_empty
_string) ... ok
test_csv_missing_header_rejected (tests.test_unit.TestCsvValidation.test_csv_missing_header_rejected) ... ok
test_csv_no_data_rows_rejected (tests.test_unit.TestCsvValidation.test_csv_no_data_rows_rejected) ... ok
test_csv_single_column_rejected (tests.test_unit.TestCsvValidation.test_csv_single_column_rejected) ... ok
test_csv_trims_cell_whitespace (tests.test_unit.TestCsvValidation.test_csv_trims_cell_whitespace) ... ok
test_csv_valid_returns_headers_and_rows (tests.test_unit.TestCsvValidation.test_csv_valid_returns_headers_and_rows) ... ok
test_auth_user_without_authorization_is_none (tests.test_unit.TestDatabaseUtilities.test_auth_user_without_authorization_is_none) .
.. ok
test_connect_enables_foreign_keys (tests.test_unit.TestDatabaseUtilities.test_connect_enables_foreign_keys) ... ok
test_init_db_creates_required_tables (tests.test_unit.TestDatabaseUtilities.test_init_db_creates_required_tables) ... ok
test_now_returns_timezone_aware_iso_timestamp (tests.test_unit.TestDatabaseUtilities.test_now_returns_timezone_aware_iso_timestamp)
 ... ok
test_owned_returns_only_matching_user_record (tests.test_unit.TestDatabaseUtilities.test_owned_returns_only_matching_user_record) .
.. ok
test_classification_accuracy_perfect (tests.test_unit.TestMetrics.test_classification_accuracy_perfect) ... ok
test_classification_accuracy_zero (tests.test_unit.TestMetrics.test_classification_accuracy_zero) ... ok
test_classification_confusion_matrix_shape (tests.test_unit.TestMetrics.test_classification_confusion_matrix_shape) ... ok
test_classification_metrics_contains_expected_keys (tests.test_unit.TestMetrics.test_classification_metrics_contains_expected_keys)
 ... ok
test_regression_error_non_negative (tests.test_unit.TestMetrics.test_regression_error_non_negative) ... ok
test_regression_metrics_contains_expected_keys (tests.test_unit.TestMetrics.test_regression_metrics_contains_expected_keys) ... ok
test_regression_perfect_error_is_zero (tests.test_unit.TestMetrics.test_regression_perfect_error_is_zero) ... ok
test_single_sample_regression_r2_is_none (tests.test_unit.TestMetrics.test_single_sample_regression_r2_is_none) ... ok
test_check_password_correct (tests.test_unit.TestPasswordUtilities.test_check_password_correct) ... ok
test_check_password_incorrect (tests.test_unit.TestPasswordUtilities.test_check_password_incorrect) ... ok
test_check_password_malformed_empty (tests.test_unit.TestPasswordUtilities.test_check_password_malformed_empty) ... ok
test_check_password_malformed_extra_separator (tests.test_unit.TestPasswordUtilities.test_check_password_malformed_extra_separator)
 ... ok
test_check_password_malformed_no_separator (tests.test_unit.TestPasswordUtilities.test_check_password_malformed_no_separator) ... o
k
test_hash_accepts_supplied_salt (tests.test_unit.TestPasswordUtilities.test_hash_accepts_supplied_salt) ... ok
test_hash_returns_salt_separator_digest (tests.test_unit.TestPasswordUtilities.test_hash_returns_salt_separator_digest) ... ok
test_hash_uses_unique_random_salts (tests.test_unit.TestPasswordUtilities.test_hash_uses_unique_random_salts) ... ok

----------------------------------------------------------------------
Ran 32 tests in 2.618s

OK
--> Unit testing: PASSED; 32 tests; 0 failures; 0 errors
```

Figure 3.1: Unit Testing Execution Results Showing 32 Successfully Passed Test Cases

Explanation: The screenshot displays the execution of the ModelLab unit testing suite. Each individual test case is marked as ok, indicating successful execution. The final execution summary confirms that all 32 test cases passed in approximately 2.618 seconds, with zero failures and zero errors.

3.2 Unit Testing Results Summary

The results obtained from the unit testing execution are summarized below.

Evaluation Parameter	Result
Total test cases executed	32
Successfully passed	32
Failed test cases	0
Errors encountered	0
Execution time	2.618 seconds
Overall execution status	PASSED

3.3 Result Analysis

The test execution achieved a 100% pass rate, demonstrating that all 32 unit test cases completed successfully. The results indicate that:

- Password utility functions performed as expected during the tested scenarios.
- CSV validation handled valid and invalid input conditions successfully.
- Classification and regression metric calculations produced the expected results.
- Database utility functions passed the implemented initialization, foreign-key, ownership, and authentication checks.

No test failures or execution errors were reported during the execution.

$$\text{Pass Rate} = \frac{32}{32} \times 100 = 100\%$$

3.4 Conclusion

The unit testing phase of ModelLab was completed successfully, with all 32 test cases passing. The execution results confirm that the individual functions and utility components covered by the unit test suite behaved as expected under the tested conditions. This establishes a foundation for proceeding with functional, integration, system, and end-to-end testing.

Approvals

Role	Name	Signature / Email	Date
Project Team Member	**Pratham Kumar** — PES2UG24CS368	prathamkumar041104@gmail.com	11-11-2026
Project Team Member	**Nirupama Jayaraman** — PES2UG24CS324	jnirupama06@gmail.com	11-11-2026
Project Team Member	**Shreyas Bellad** — PES2UG25CS824	shreyasbellad2006@gmail.com	11-11-2026
Project Team Member	**Parinitha Acharya** — PES2UG24CS339	parinitha.acharya2006@gmail.com	11-11-2026
Course Coordinator	**Sheela Devi M.**	sheelam@pes.edu	—

THANK_YOU